



Shailesh Yadav-da

Data Analyst

**Seaborn— COMPLETE DATA
VISUALIZATION WORKBOOK**



Welcome To My Post

Scroll More >

Seaborn Library

Seaborn is a [Python](#) data **visualization** library used for creating statistical graphics. It helps data analysts visualize data in a more beautiful, clean, and informative way.

Seaborn is built on top of Matplotlib, which means it uses Matplotlib internally but provides a simpler and more advanced interface.

1. What is Seaborn?

Seaborn is a high-level visualization library in Python that is mainly used for statistical data visualization.

In simple words:

- Matplotlib → basic plotting library
- Seaborn → **advanced** and better-looking plots built on Matplotlib

Seaborn automatically applies better colors, styles, and layouts, making charts look more professional.

2. Why Do Data Analysts Use Seaborn?

Seaborn is popular among data analysts because it helps in:

1. Creating statistical visualizations
2. Making beautiful default charts
3. Working directly with Pandas DataFrames
4. Showing relationships between variables

Common analysis tasks include:

- Understanding data distribution
- Comparing categories
- Finding relationships between variables
- Performing correlation analysis

What is a Line Plot?

A Line Plot displays data points connected by straight lines.

It is mainly used when:

- Data changes over time
- Data follows a continuous sequence
- We want to observe trends

Example:

- Monthly sales
- Website traffic over days
- Temperature changes
- Stock price movement

2 How a Line Plot Works

A line plot uses two variables:

- **X-axis** → Independent variable (time, index, sequence)
- **Y-axis** → Dependent variable (value that changes)

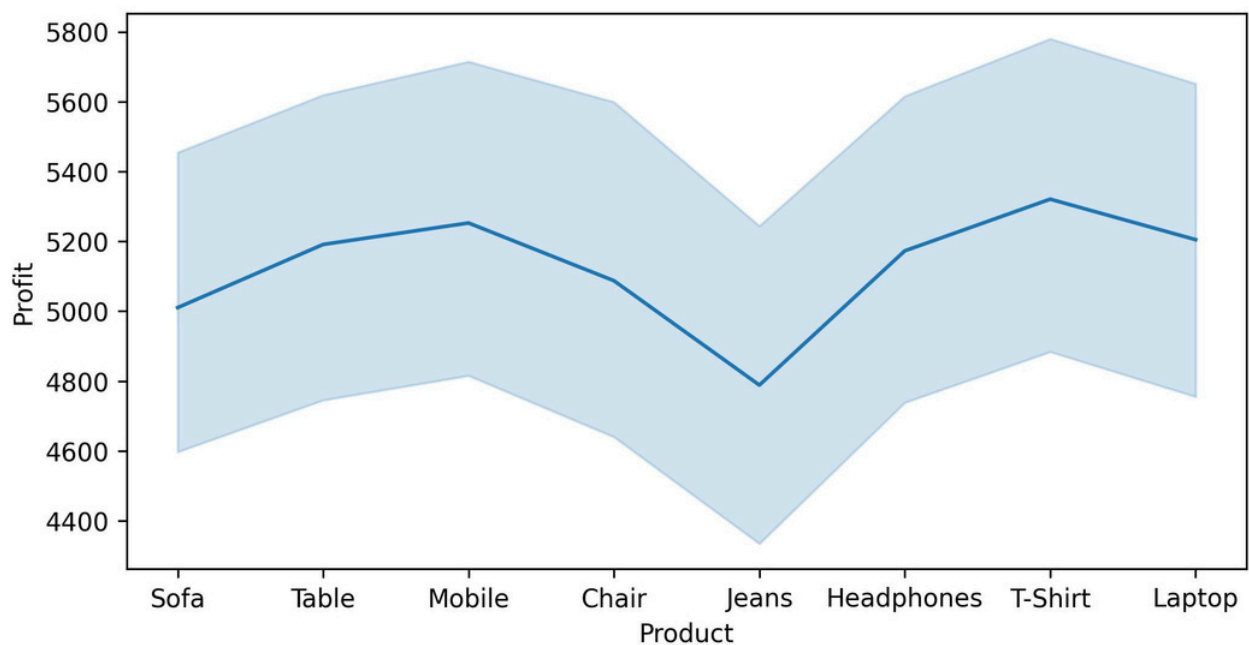
The points are connected with lines to show the trend.

3 Basic Syntax

```
sns.lineplot(x= , y= , data= )
```

4 Example – Creating a Line Plot

```
plt.figure(figsize=(8,4))  
sns.lineplot(x="Product",y="Profit",data=df)  
plt.show()
```



What is a Histogram?

A Histogram divides continuous **numerical data** into intervals called bins and displays the frequency of values in each bin.

When to Use Histogram

Data analysts use histograms when they want to understand:

- Data distribution
- Data spread
- Whether data is normal, skewed, or uniform

Common examples:

- Salary distribution
- Age distribution
- Exam score distribution
- Product price distribution

3 Basic Syntax

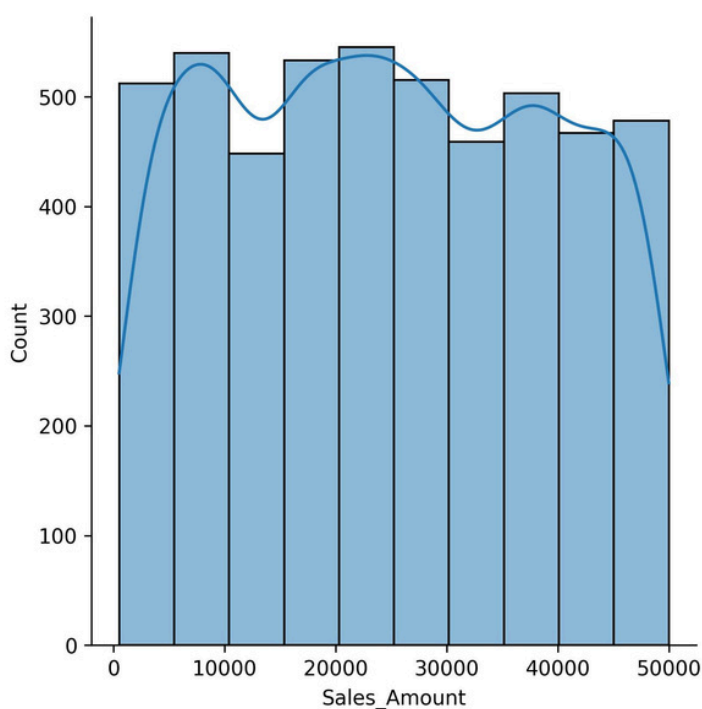
```
sns.histplot(data= , x= )
```

Parameters:

- data → dataset / DataFrame
- x → numerical column to analyze

4 Basic Example

```
sns.displot(data=df, x="Sales_Amount",  
bins=10, kde=True,  
plt.show())
```



3 What is a Bar Plot?

A Bar Plot shows the relationship between a **categorical** variable and a **numerical** variable.

2 When to Use a Bar Plot

Data analysts use bar plots when they want to:

- Compare categories
- Show **aggregated** values (mean, sum, count)
- Present business comparisons

Examples:

- Sales by product
- Revenue by region
- Average salary by department
- Orders by category

3 Basic Syntax

```
sns.barplot(x= , y= , data= )
```

Parameters:

- x → categorical column
- y → numerical column
- data → dataset

4 Basic Example

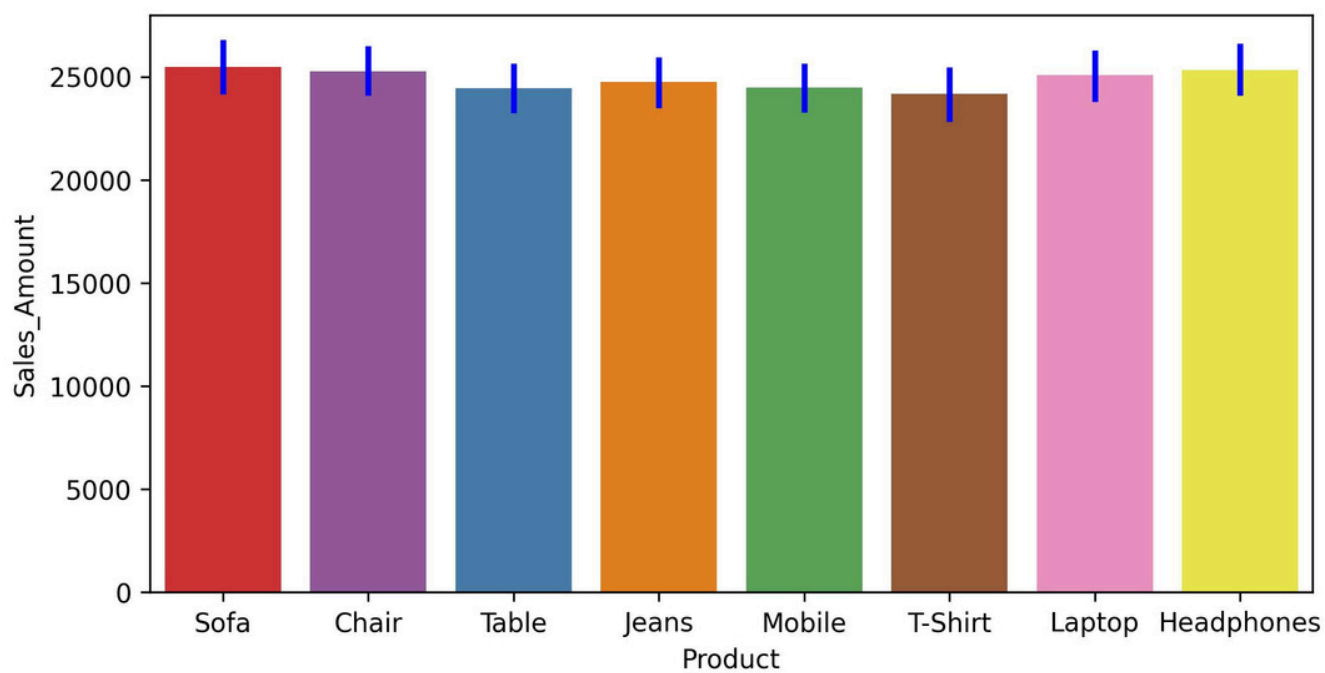
```
sns.barplot(x="Product", y="Sales_Amount",  
data=df)  
plt.show()
```

Result:

- X-axis → Products
- Y-axis → Sales
- Bars show sales comparison

Advance Examples:

```
plt.figure(figsize=(8,4))
order_1 =
["Sofa","Chair","Table","Jeans","Mobile","T-
Shirt","Laptop","Headphones"]
sns.barplot(
    x="Product",
    y="Sales_Amount",
    data=df,
    hue="Product",
    palette="Set1",
    legend=False,
    order=order_1,
    errcolor= "b"
)
plt.show()
```



4 What is a Scatter Plot?

A Scatter Plot is used to visualize the relationship **between two numerical variables**.

Each point represents one record in the dataset.

Example in your dataset:

- Sales_Amount
- Profit

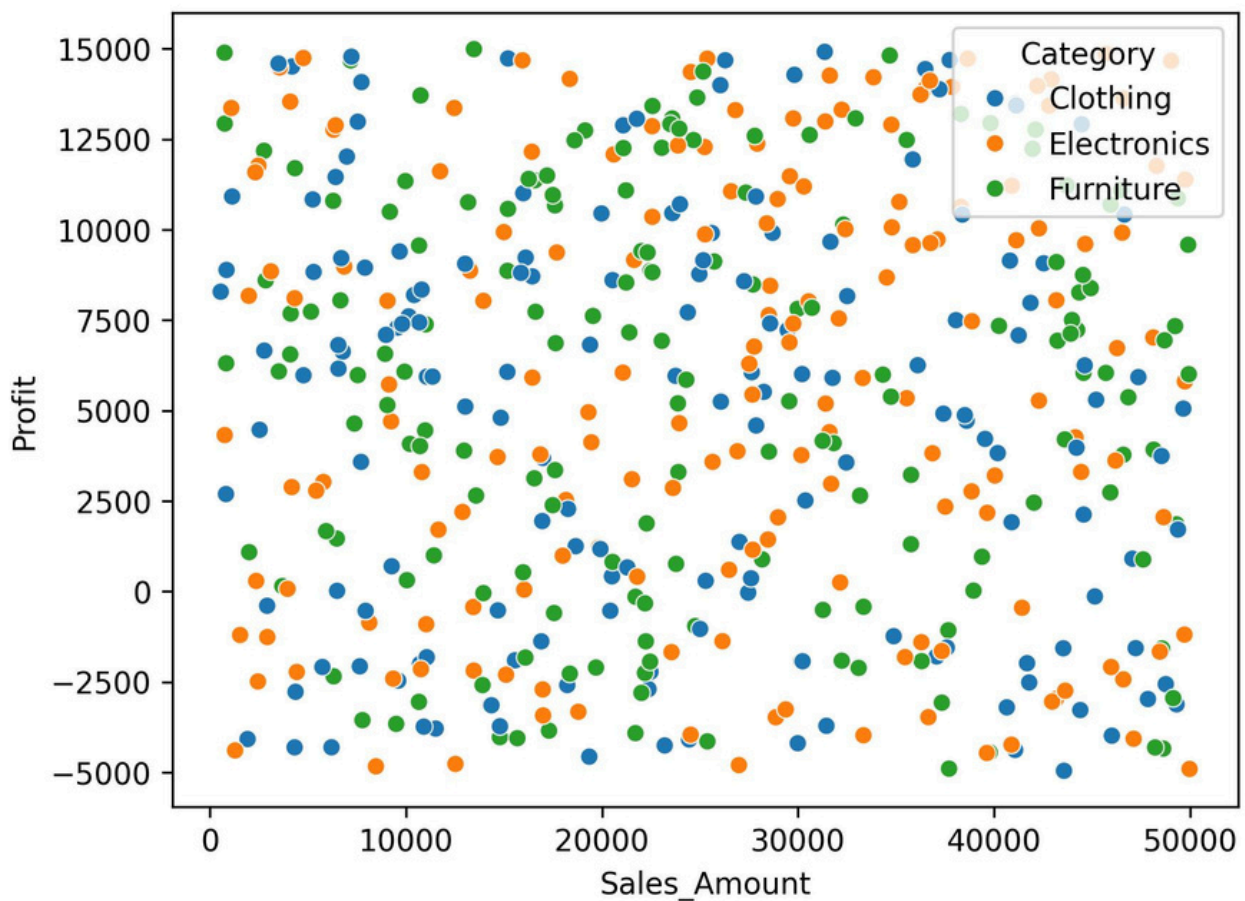
Each order will appear as one dot.

2 Basic Syntax

```
sns.scatterplot(x= , y= , data= )
```

3 Example:

```
sns.scatterplot(  
    data=df.head(500),  
    x="Sales_Amount",  
    y="Profit",  
    hue="Category",  
)  
plt.show()
```



5 What is a Heatmap?

A Heatmap is a graphical representation of data where values are represented using colors.

- Higher values → darker color
- Lower values → lighter color

It is mainly used to visualize correlation between **numerical variables**.

2 Why Data Analysts Use Heatmaps

Heatmaps help analysts quickly see:

- Which variables are strongly related
- Which variables have weak relationships
- Whether variables have positive or negative correlation

3 Step 1 – Create Correlation Matrix

First calculate correlation.

```
corr = df[["Sales_Amount","Quantity","Profit"]].corr()
```

4 Step 2 – Create Heatmap

```
import seaborn as sns  
import matplotlib.pyplot as plt
```

```
corr = df[["Sales_Amount","Quantity","Profit"]].corr()
```

```
sns.heatmap(corr)
```

```
plt.title("Correlation Heatmap")  
plt.show()
```

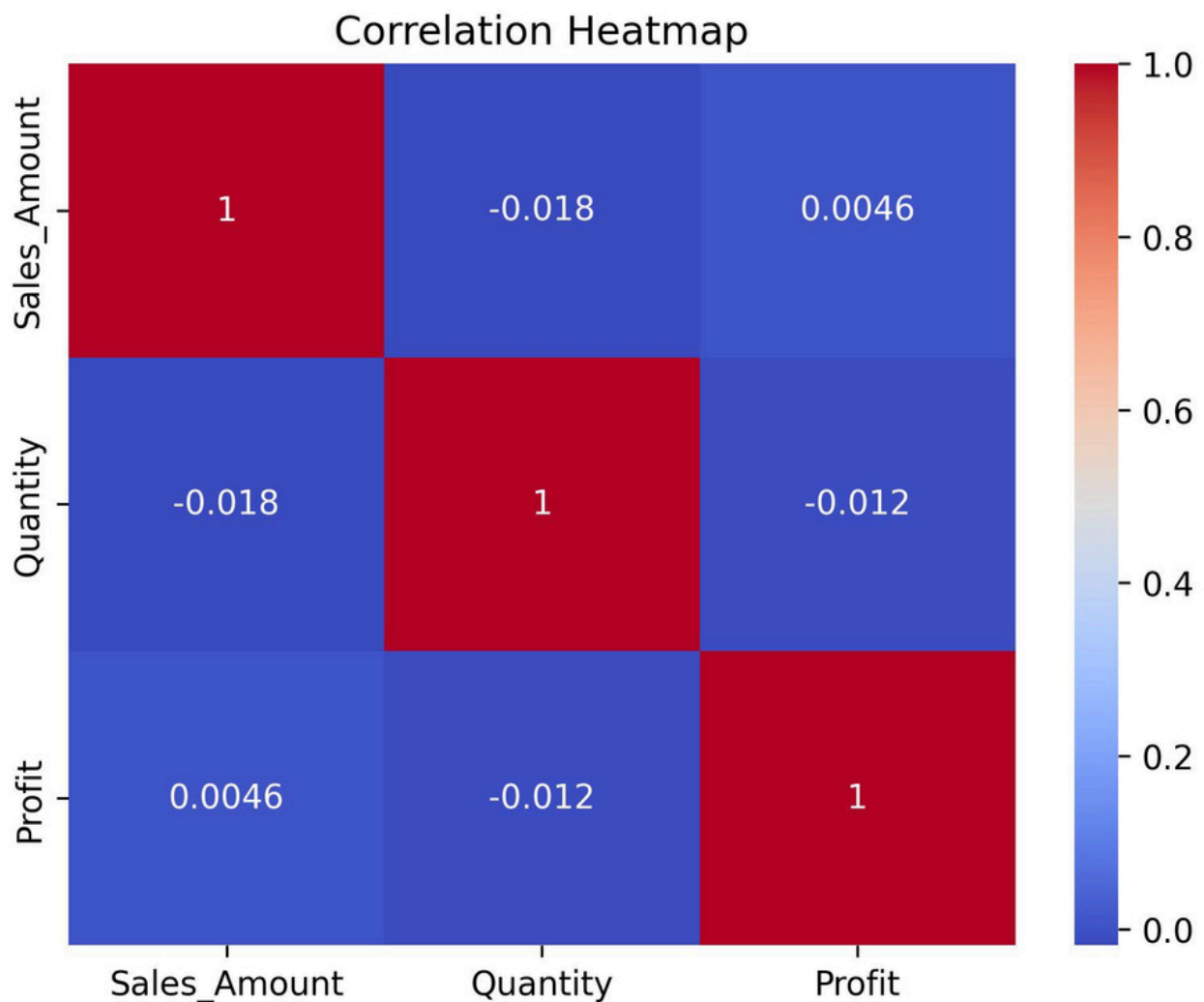
This creates the **basic** heatmap.

5 Professional Heatmap

```
sns.heatmap(corr,annot=True,cmap="coolwarm")
```

```
plt.title("Correlation Heatmap")
```

```
plt.show()
```



6 How to Interpret the Heatmap

Correlation values range:

Value	Meaning
1	Perfect positive correlation
0	No correlation
-1	Perfect negative correlation

7 Real Analyst Insight

Using this heatmap you can quickly identify:

- ✓ Does increasing Sales increase Profit
- ✓ Does selling more Quantity increase profit
- ✓ Which variable impacts business most

Plot 6 — Count Plot

What is a Count Plot?

A Count Plot shows the number of observations in each category.

It simply counts how many times each category appears in a column.

When to Use Count Plot

Data analysts use count plots when they want to analyze:

- Category frequency
- Data distribution across categories
- Dataset balance

Examples:

- Orders by region
- Orders by product category
- Customers by gender
- Transactions by payment type

3 Basic Syntax

```
sns.countplot(x= , data= )
```

Parameters:

- x → categorical column
- data → dataset

4 Example:

Orders by Category

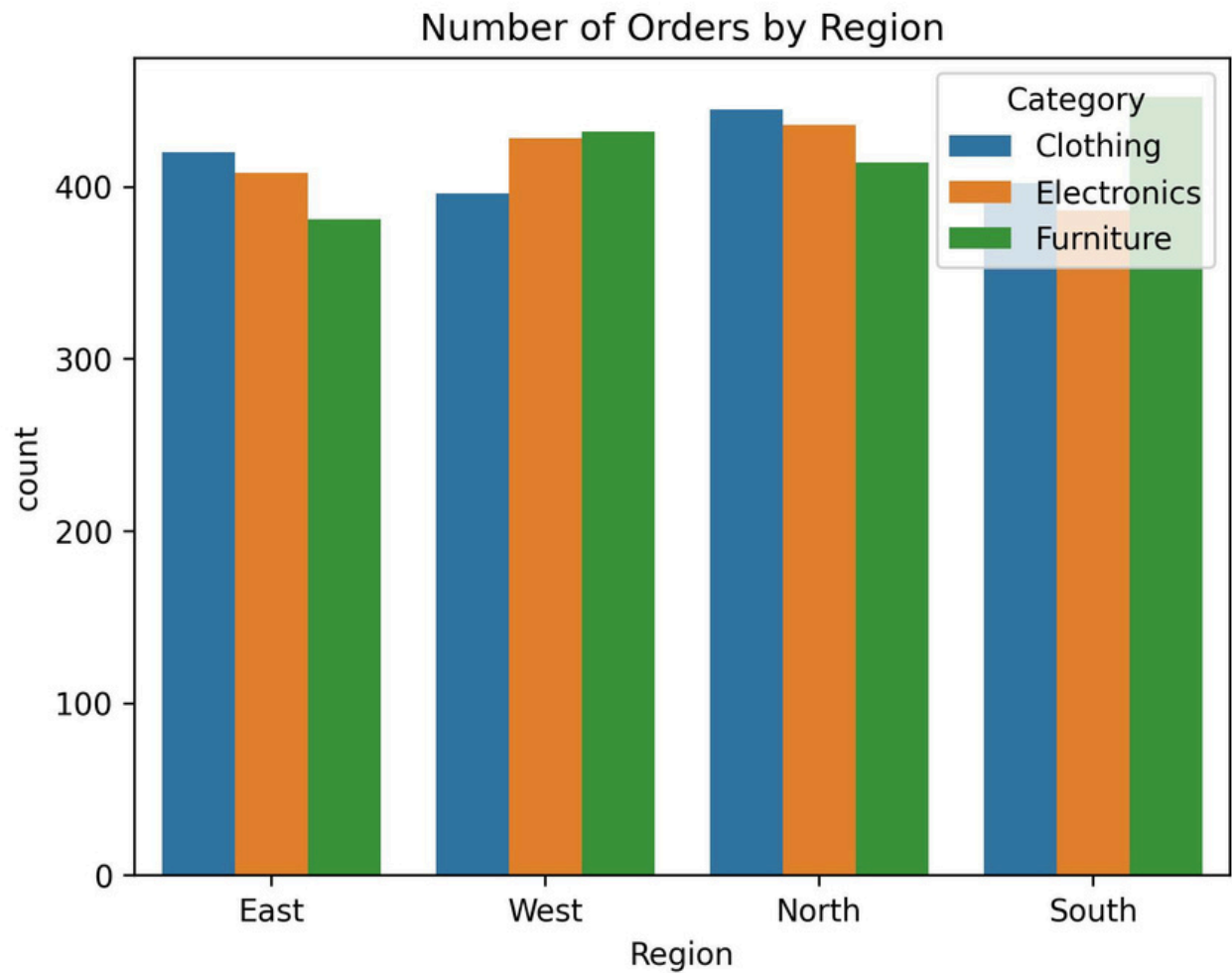
```
sns.countplot(  
    data=df,  
    x="Category"  
)  
  
plt.title("Number of Orders by Category")  
plt.show()
```

Result:

- X-axis → Category
- Y-axis → Number of orders

5 Count Plot Advance:

```
sns.countplot(  
    data=df,  
    x="Region",  
    hue = "Category"  
)  
  
plt.title("Number of Orders by Region")  
plt.show()
```



Very Important Difference

Bar Plot:

Average Sales by Category

Count Plot:

Number of Orders by Category

Plot 7 — Violin Plot

What is a Violin Plot?

A Violin Plot shows the distribution of **numerical** data across different **categories**.

It is basically a combination of:

- Box Plot (median + quartiles)
- Kernel Density Plot (KDE) (data distribution shape)

Because of this, it looks like a violin instrument shape.

2 Why Analysts Use Violin Plots

A violin plot helps analysts see:

- Data spread
- Data density
- Median values
- Differences between categories

So it gives more information than a box plot.

3 Basic Syntax

```
sns.violinplot(x= , y= , data= )
```

Parameters:

- x → categorical variable
- y → numerical variable
- data → dataset

4 Example:

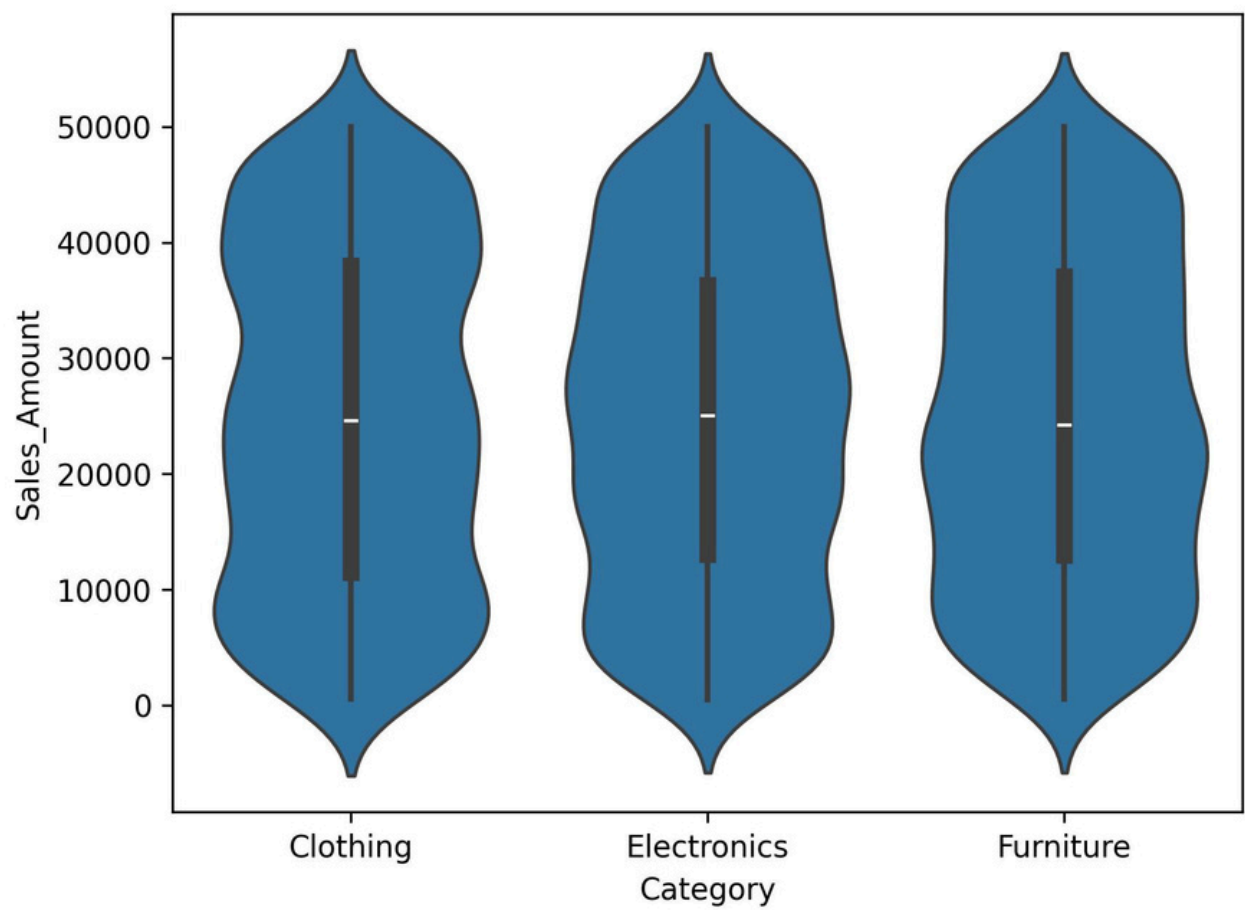
Sales Distribution by Category

```
sns.violinplot(  
    data=df,  
    x="Category",  
    y="Sales_Amount"  
)
```

```
plt.title("Sales Distribution by Category")  
plt.show()
```

Result:

- X-axis → Category
- Y-axis → Sales Amount
- Shape shows distribution of sales



Plot 8 — Pair Plot

What is a Pair Plot?

A Pair Plot shows the relationship between multiple numerical variables at once.

It creates a grid of plots:

- Diagonal → Histogram (distribution)
- Off-diagonal → Scatter plots (relationships)

 **In one plot, you get:**

- Distribution of each variable
- Relationship between every pair of variables

Why Pair Plot is Important

Pair plot helps analysts:

- Detect correlation
- Identify patterns
- Find outliers
- Understand data distribution

 It is like complete EDA in one visualization

Basic Syntax

```
sns.pairplot(data= )
```

4 Important Parameters (Must Know)

vars

Select specific columns

```
vars=["Sales_Amount","Profit"]
```

hue

Add category grouping

```
hue="Category"
```

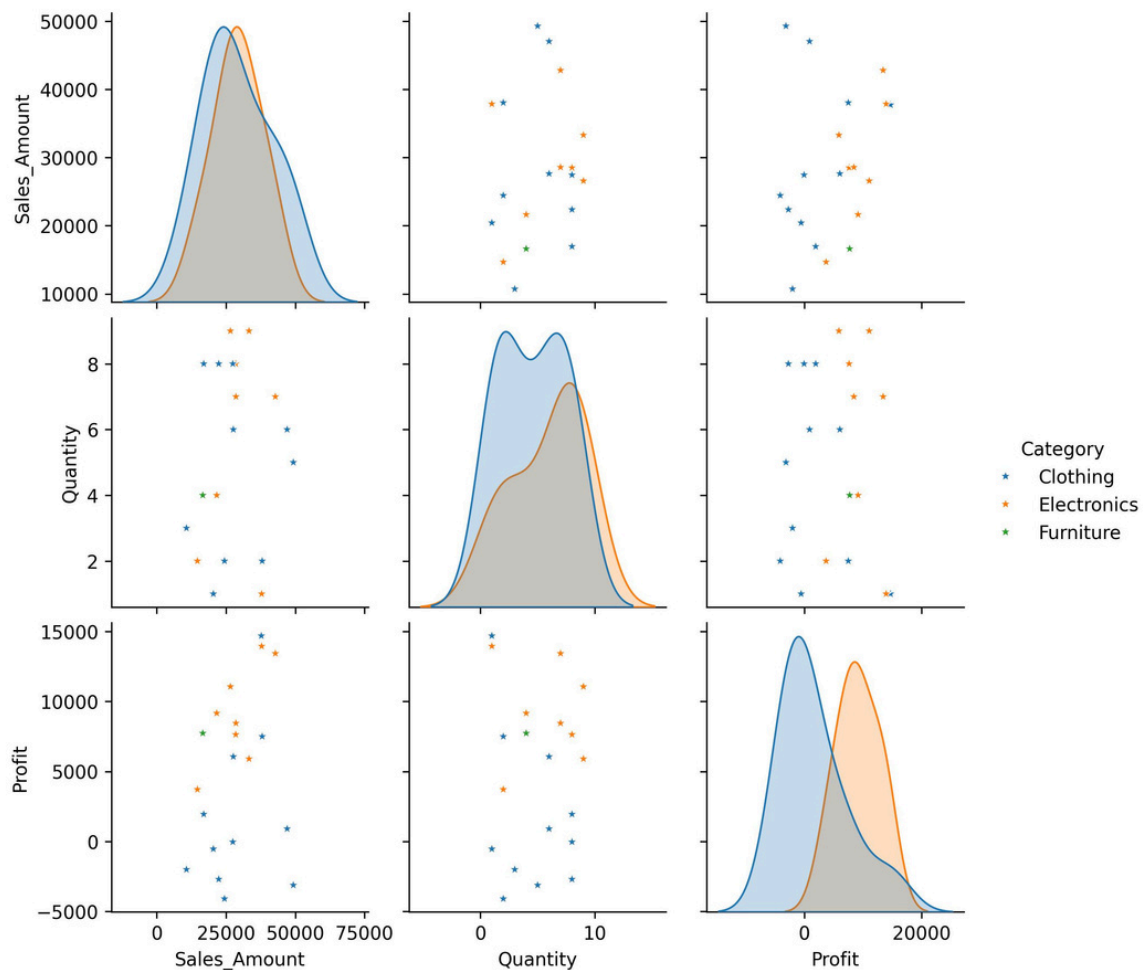
diag_kind

Change diagonal plot

```
diag_kind="kde"
```

Example:

```
sns.pairplot(  
    df.head(20),  
    vars=["Sales_Amount", "Quantity", "Profit"],  
    hue="Category",  
    diag_kind="kde",  
    markers = "*" )  
plt.show()
```



Plot 9 – Strip Plot

What is a Strip Plot?

A Strip Plot shows all individual data points for a **categorical** variable.

 It is similar to a scatter plot but:

- One axis = categorical
- One axis = numerical

Each point represents one observation (row).

Why Use Strip Plot?

Strip plot helps analysts:

- See actual data points
- Detect outliers
- Understand data spread
- Compare categories

 Unlike box/violin plots, it shows raw data (no aggregation)

3 Basic Syntax

```
sns.stripplot(x= , y= , data= )
```

4 Example:

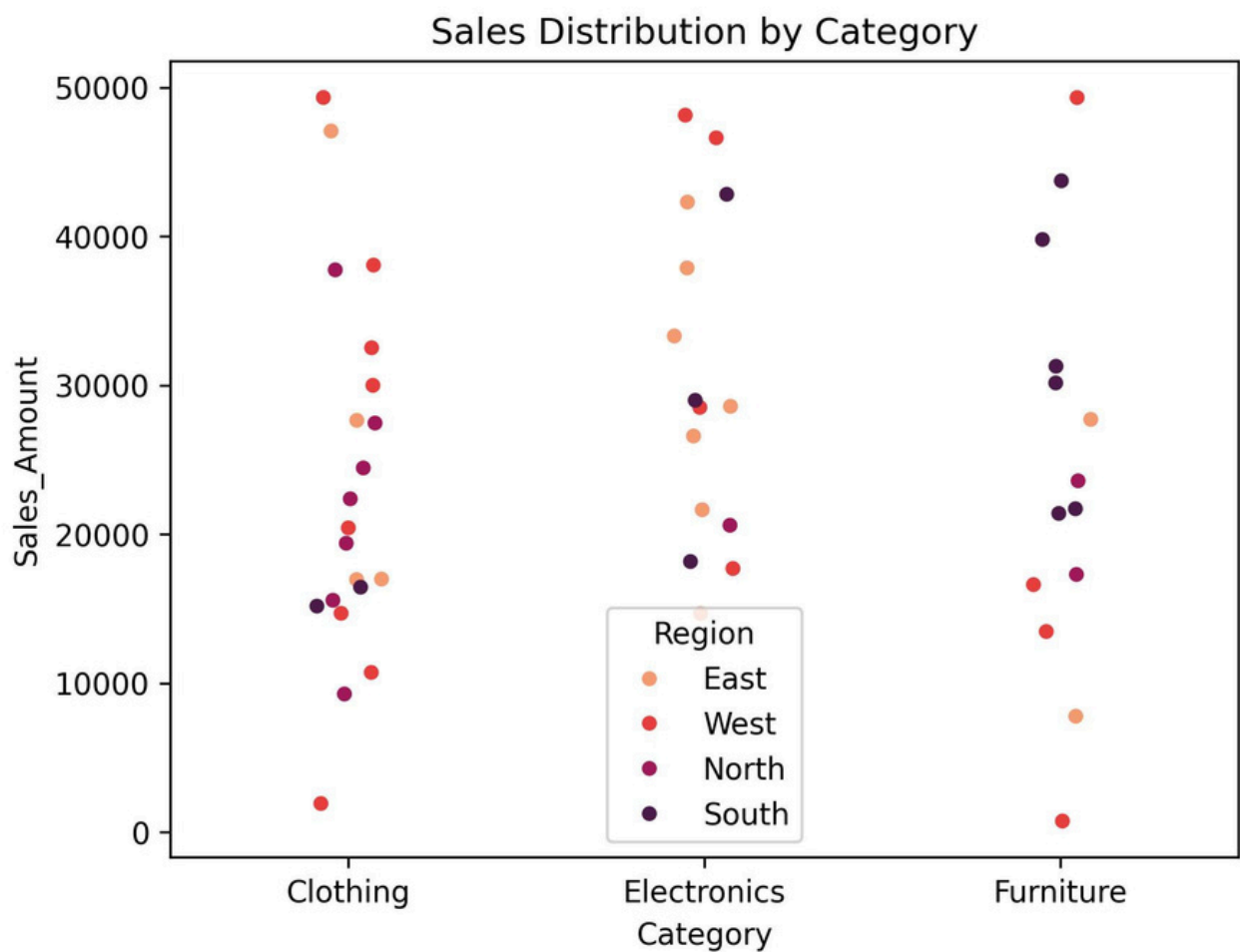
Sales by Category

```
sns.stripplot(  
    data=df.head(50),  
    x="Category",  
    y="Sales_Amount",  
    hue="Region",  
    palette = "rocket_r",  
    jitter=True  
)  
plt.title("Sales Distribution by Category")  
plt.show()
```

Important Problem – Overlapping Points

Like scatter plot, strip plot can also become messy.

✓ Solution → use jitter



Plot 10 — Box Plot

What is a Box Plot?

A Box Plot (also called Box-and-Whisker Plot) shows:

- Minimum value
- Maximum value
- Median
- Quartiles (Q1, Q3)
- Outliers

 It summarizes the distribution of data in a compact way.

Core Concept (Very Important)

A box plot is based on quartiles:

- Q1 (25%) → lower quartile
- Q2 (50%) → median
- Q3 (75%) → upper quartile

 The box = middle 50% of data (IQR)

Formula (Important for Interviews)

 Outliers are detected using:

- Lower bound = $Q1 - 1.5 \times IQR$
- Upper bound = $Q3 + 1.5 \times IQR$

3 Why Analysts Use Box Plot

Box plot helps in:

- ✓ Detecting outliers
- ✓ Understanding data spread
- ✓ Comparing categories
- ✓ Finding skewness

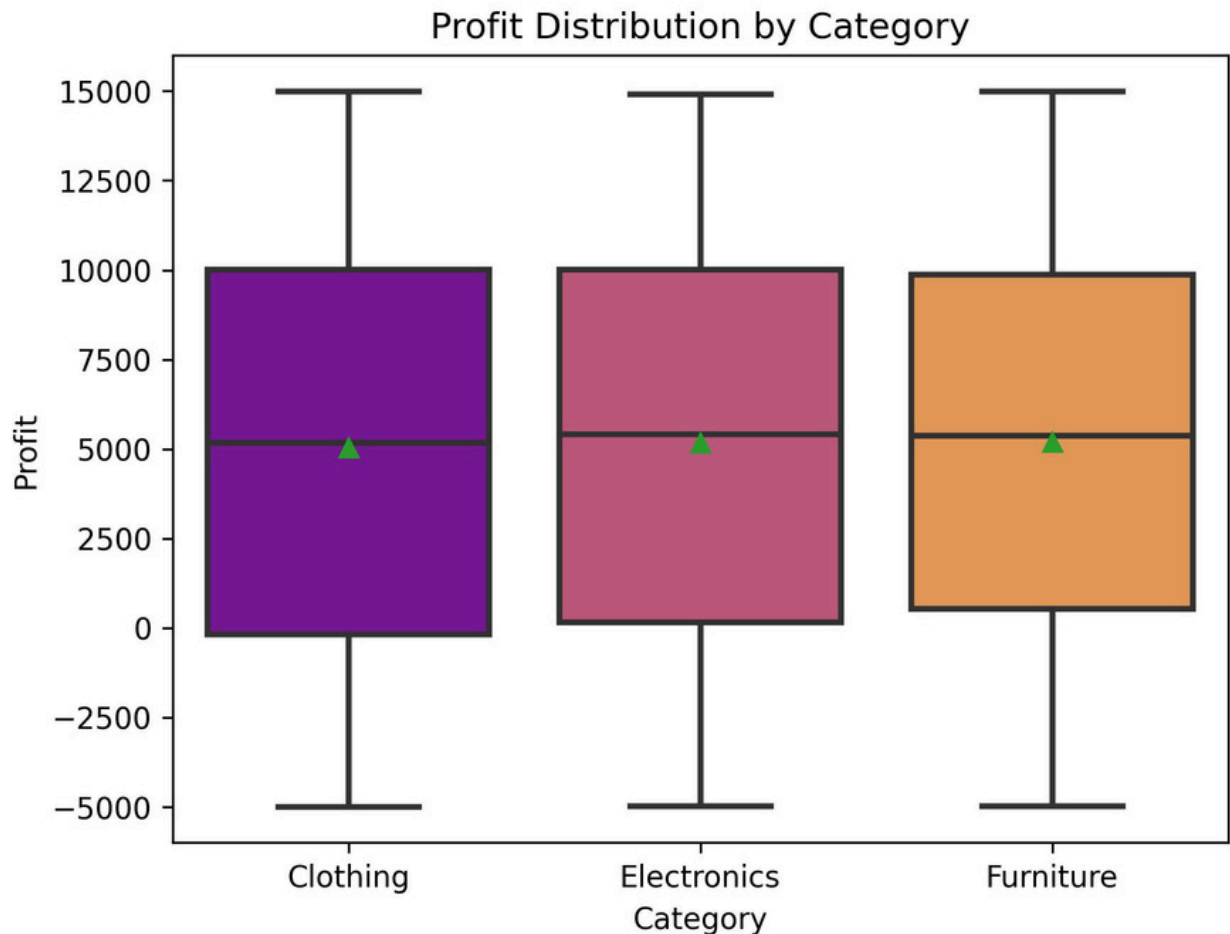
4 Basic Syntax

```
sns.boxplot(x= , y= , data= )
```

5 Example:

Profit Distribution by Category

```
sns.boxplot(  
    data=df,  
    x="Category",  
    y="Profit",  
    showmeans = True,  
    linewidth = 2,  
    palette = "plasma"  
)  
plt.title("Profit Distribution by  
Category")  
plt.show()
```



How to Read Box Plot (Very Important)

Each part means:

- Middle line → Median
- Box → Middle 50% (Q1 to Q3)
- Whiskers → Range
- Dots outside → Outliers

⚠ Important Reality (Very Important)

👉 `factorplot()` is deprecated

(removed) in modern Seaborn.

✓ Old name → `factorplot()`

✓ New name → `catplot()`

Plot — Factor Plot (Catplot)

1 What is Factor Plot?

A Factor Plot (`catplot`) is used to visualize relationships between **categorical** and **numerical** data.

👉 It is a figure-level function that can create multiple types of plots:

- Bar plot
- Count plot
- Box plot
- Violin plot
- Strip plot

👉 Think of it as a wrapper for all categorical plots.

2 Basic Syntax

```
sns.catplot(x= , y= , data= , kind= )
```

3 Example:

Bar Plot using Catplot

```
sns.catplot(  
    data=df,  
    x="Category",  
    y="Sales_Amount",  
    kind="bar"  
)
```

4 Different Types (Very Important)

1 Count Plot

```
sns.catplot(  
    data=df,  
    x="Category",  
    kind="count"  
)
```

2 Box Plot

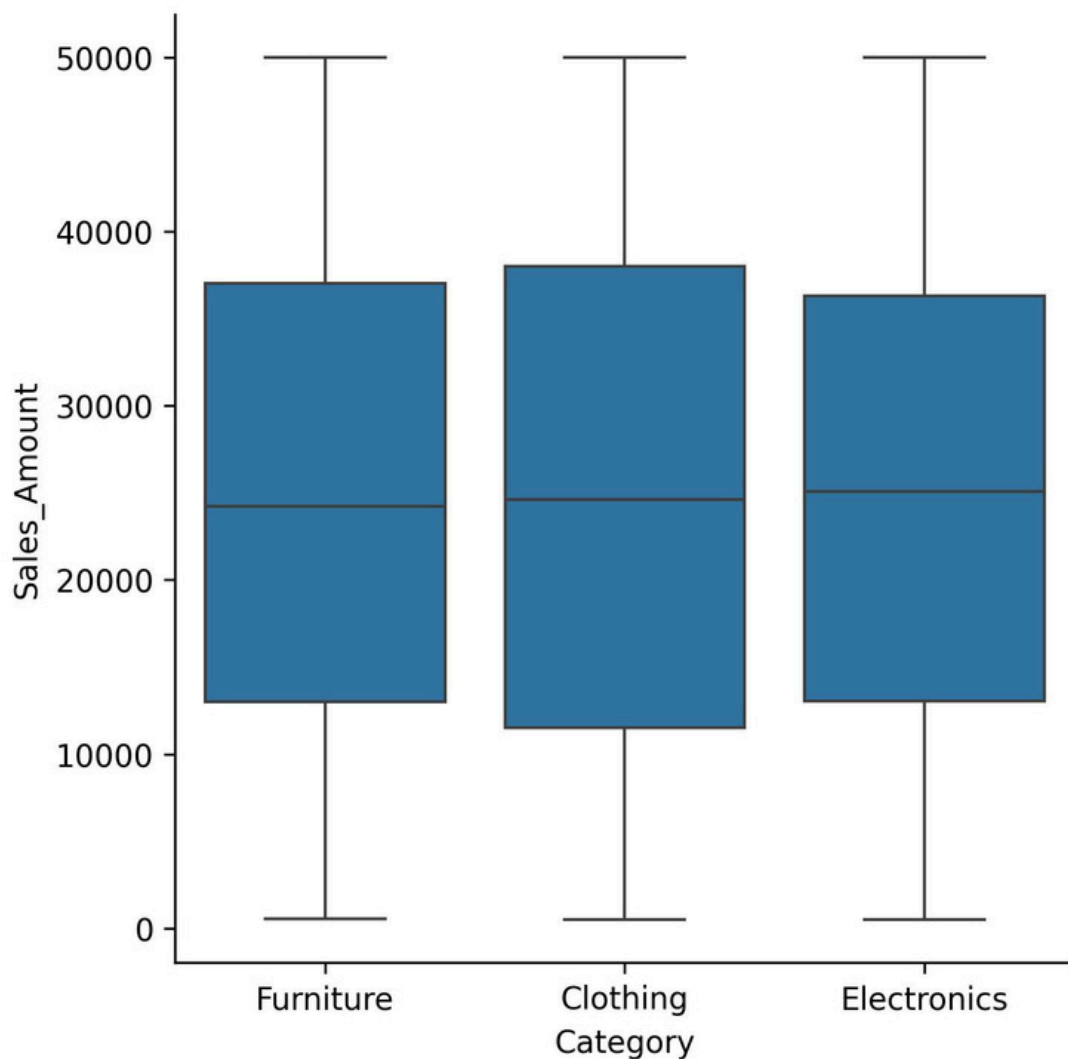
```
sns.catplot(  
    data=df,  
    x="Category",  
    y="Profit",  
    kind="box"  
)
```

3 Violin Plot

```
sns.catplot(  
    data=df,  
    x="Category",  
    y="Profit",  
    kind="violin"  
)
```

Perfect Example:

```
sns.catplot(  
    data=df,  
    x="Category",  
    y="Sales_Amount",  
    kind = "box",  
    order = ["Furniture","Clothing","Electronics"]  
)  
plt.show()
```



Styling in Seaborn

1 What is Plot Styling?

Plot Styling ka matlab hai:

- Colors change karna
- Background improve karna
- Fonts aur titles set karna
- Layout clean banana

 **Goal:** Readable + Professional Visualization

2 Default Style vs Styled Plot

Without styling:

- Plain graph
- Basic look

With styling:

- Clean
- Attractive
- Dashboard-ready

3 Seaborn Built-in Styles

Seaborn me kuch predefined styles hote hain:

```
sns.set_style("white")  
sns.set_style("dark")  
sns.set_style("whitegrid")  
sns.set_style("darkgrid")  
sns.set_style("ticks")
```

👉 **Most used:**

- ✓ "whitegrid" (best for analysis)
- ✓ "darkgrid"

4 Color Palettes (Very Important)

Colors change karne ke liye:

```
sns.set_palette("Set2")
```

Popular palettes:

- "Set1"
- "Set2"
- "pastel"
- "coolwarm"
- "viridis"

Figure Size Control

Plot ka size control:

```
plt.figure(figsize=(8,5))
```


6 Titles and Labels (Must for Professional Look)

```
plt.title("Sales Analysis by Category")  
plt.xlabel("Category")  
plt.ylabel("Sales Amount")
```

7 Grid Control

```
sns.set_style("whitegrid")
```

or remove grid:

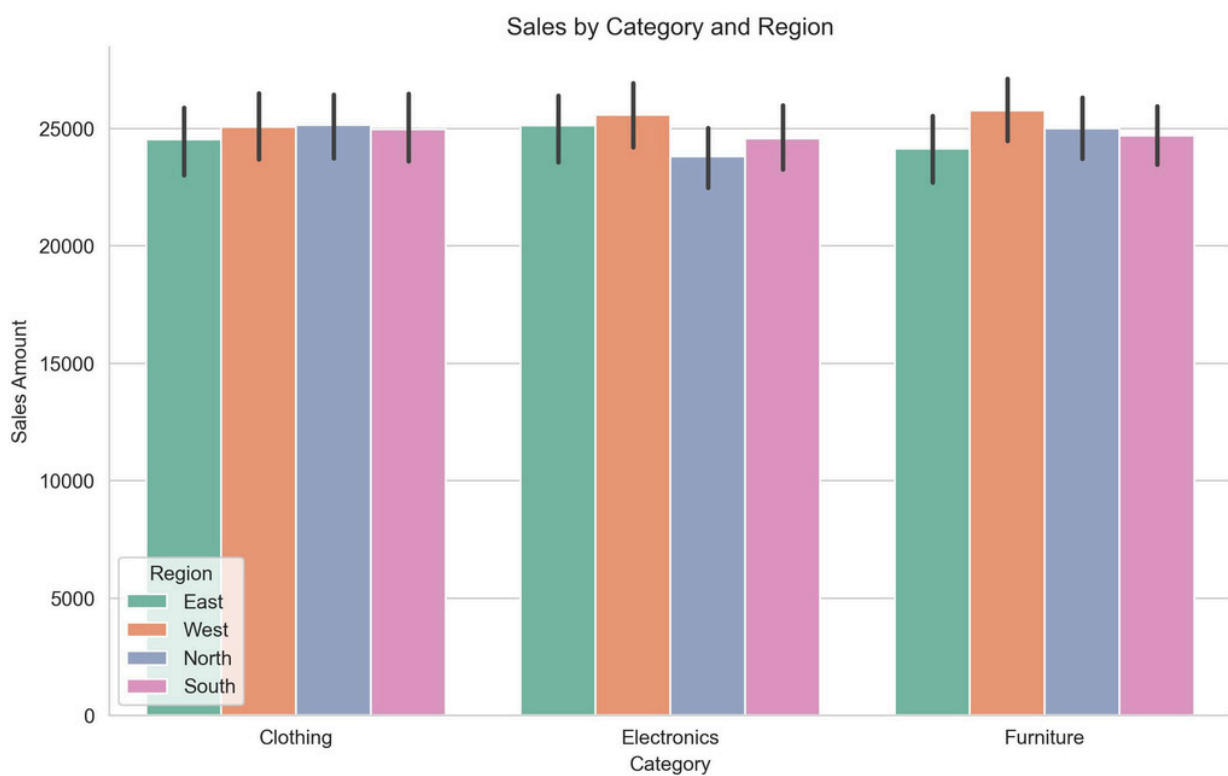
```
sns.set_style("white")
```

8 Remove Extra Borders (Advanced Clean Look)

`sns.despine()`

👉 Removes top and right borders

9 Complete Professional Example



```
sns.set_style("whitegrid")  
sns.set_palette("Set2")
```

```
plt.figure(figsize=(10,6))
```

```
sns.barplot(  
    data=df,  
    x="Category",  
    y="Sales_Amount",  
    hue="Region"  
)
```

```
plt.title("Sales by Category and Region")  
plt.xlabel("Category")  
plt.ylabel("Sales Amount")
```

```
sns.despine()
```

```
plt.show()
```



Multiple Plots — FacetGrid

1 What is FacetGrid?

FacetGrid allows you to create multiple plots (subplots) based on categories.



It splits your dataset into subsets and draws same plot for each subset.

2 Why Use FacetGrid?

FacetGrid helps you:

- ✓ Compare data across categories
- ✓ Build dashboard-like visuals
- ✓ Perform group-wise analysis
- ✓ Improve clarity in large datasets

3 Basic Concept

Instead of one plot:

👉 You get multiple small plots (grid format)

Example:

- One plot for each Region
- One plot for each Category

4 Basic Syntax

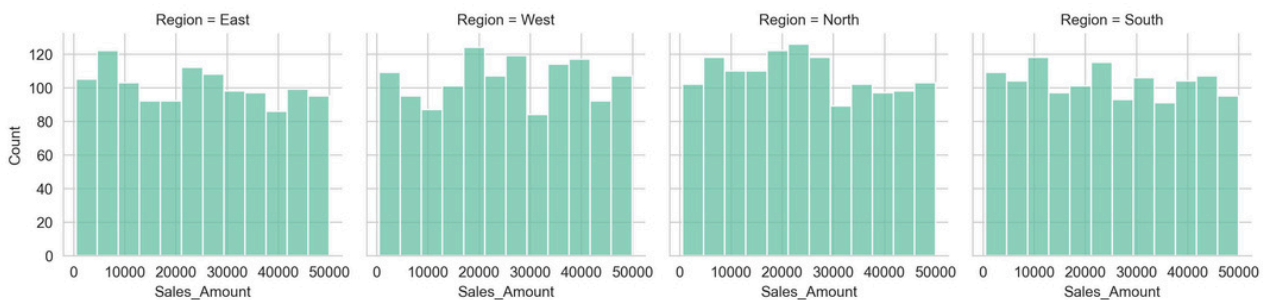
```
g = sns.FacetGrid(data= , col= )  
g.map( , )
```

5 Example:

```
g = sns.FacetGrid(df, col="Region")
```

```
g.map(sns.histplot, "Sales_Amount")
```

```
plt.show()
```

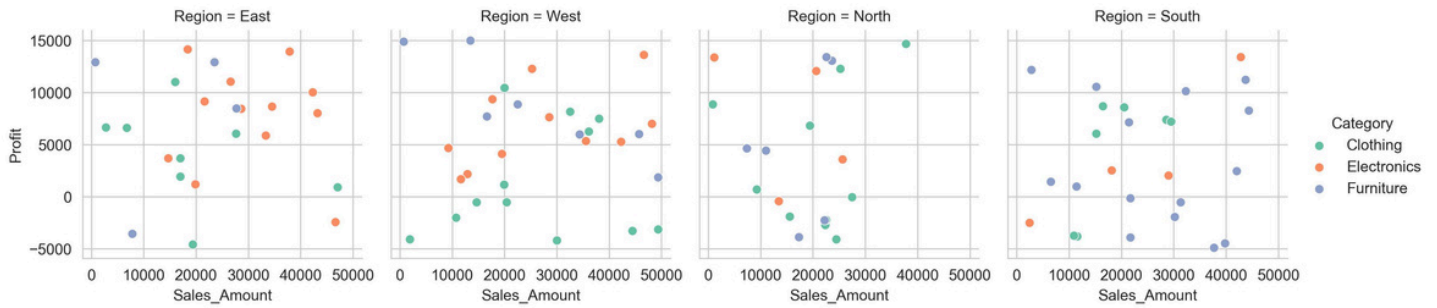


```
g = sns.FacetGrid(df.head(100), col="Region",  
hue="Category")
```

```
g.map(sns.scatterplot, "Sales_Amount", "Profit")
```

```
g.add_legend()
```

```
plt.show()
```



Real Data Analyst Insight

FacetGrid helps answer:

- ✓ Which region performs best?
- ✓ Which category behaves differently?
- ✓ Sales patterns across regions

Data Speaks, We Just Listen

Thank you for being part of this data journey.

Name. **Shailesh Yadav**

GitHub link

github.com/shailesh-yadav-da

site:

<https://sites.google.com/view/shailesh-yadav-da/home>